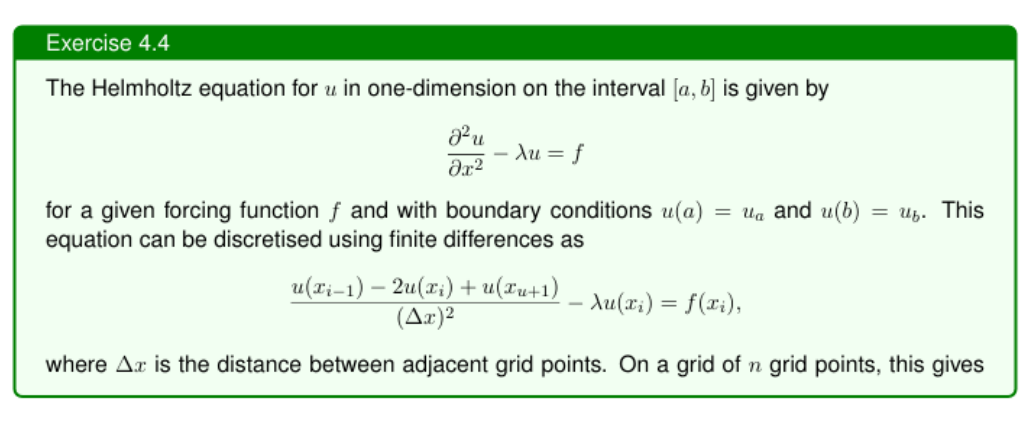
Ex 4-4 symmetric

1/30/24 08:05



Exercise 4.4

The Helmholtz equation for u in one-dimension on the interval $[a, b]$ is given by

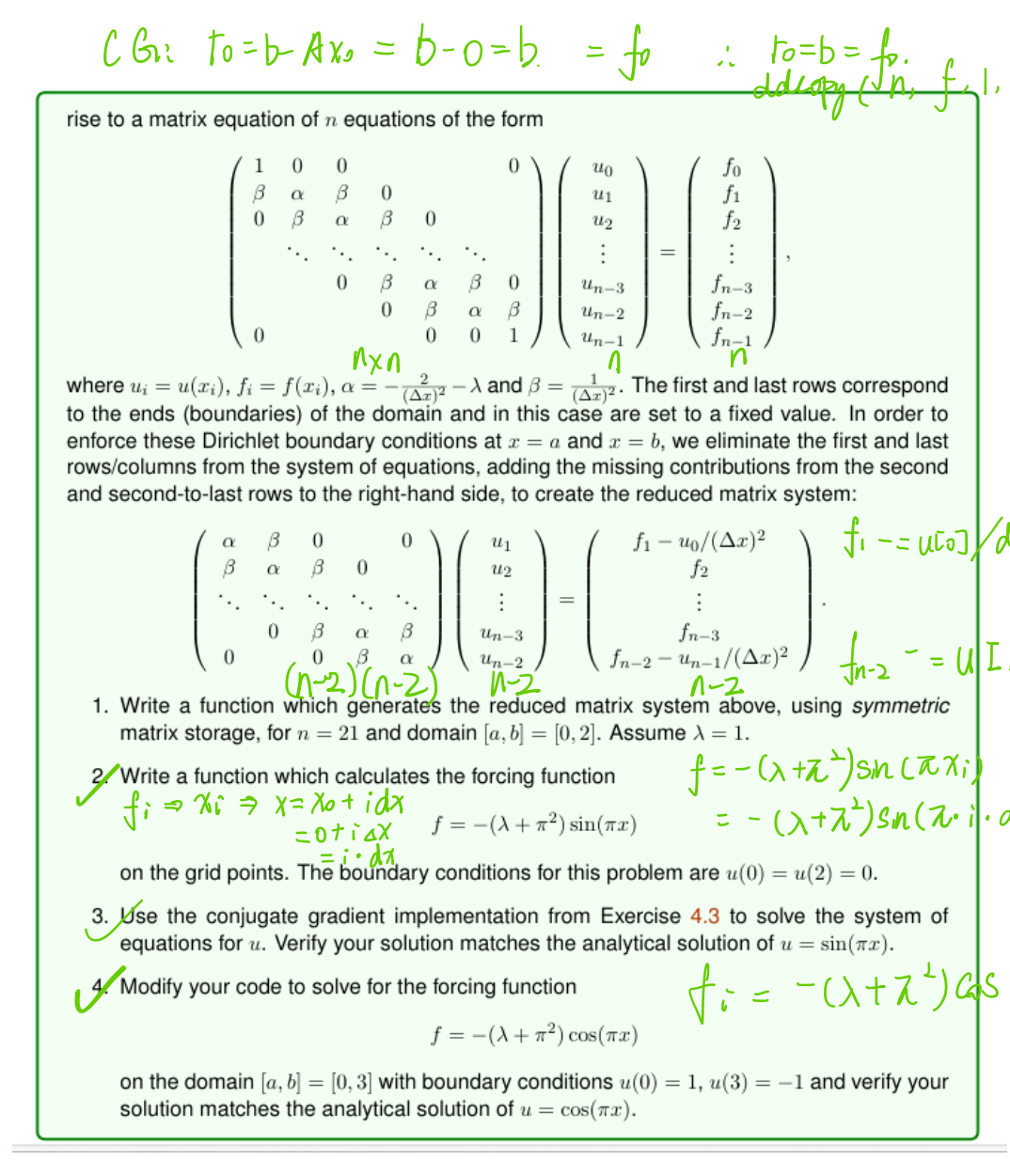
$$\frac{\partial^2 u}{\partial x^2} - \lambda u = f$$

for a given forcing function f and with boundary conditions $u(a) = u_a$ and $u(b) = u_b$. This equation can be discretised using finite differences as

$$\frac{u(x_{i-1}) - 2u(x_i) + u(x_{u+1})}{(\Delta x)^2} - \lambda u(x_i) = f(x_i),$$

where Δx is the distance between adjacent grid points. On a grid of n grid points, this gives

CG: $r_0 = b - Ax_0 = b - 0 = b = f_0$ ∴ $r_0 = b = f_0$. ddcopy(n, f, 1, r, 1).



rise to a matrix equation of n equations of the form

$$\underbrace{\begin{pmatrix} 1 & 0 & 0 & & & & 0 \\ \beta & \alpha & \beta & 0 & & & \\ 0 & \beta & \alpha & \beta & 0 & & \\ & \ddots & \ddots & \ddots & \ddots & \ddots & \\ & & 0 & \beta & \alpha & \beta & 0 \\ & & & 0 & \beta & \alpha & \beta \\ 0 & & & & 0 & 0 & 1 \end{pmatrix}}_{n \times n} \underbrace{\begin{pmatrix} u_0 \\ u_1 \\ u_2 \\ \vdots \\ u_{n-3} \\ u_{n-2} \\ u_{n-1} \end{pmatrix}}_{n} = \underbrace{\begin{pmatrix} f_0 \\ f_1 \\ f_2 \\ \vdots \\ f_{n-3} \\ f_{n-2} \\ f_{n-1} \end{pmatrix}}_{n},$$

where $u_i = u(x_i)$, $f_i = f(x_i)$, $\alpha = -\frac{2}{(\Delta x)^2} - \lambda$ and $\beta = \frac{1}{(\Delta x)^2}$. The first and last rows correspond to the ends (boundaries) of the domain and in this case are set to a fixed value. In order to enforce these Dirichlet boundary conditions at $x = a$ and $x = b$, we eliminate the first and last rows/columns from the system of equations, adding the missing contributions from the second and second-to-last rows to the right-hand side, to create the reduced matrix system:

$$\underbrace{\begin{pmatrix} \alpha & \beta & 0 & & 0 \\ \beta & \alpha & \beta & 0 & \\ \ddots & \ddots & \ddots & \ddots & \ddots \\ & 0 & \beta & \alpha & \beta \\ 0 & & 0 & \beta & \alpha \end{pmatrix}}_{(n-2)(n-2)} \underbrace{\begin{pmatrix} u_1 \\ u_2 \\ \vdots \\ u_{n-3} \\ u_{n-2} \end{pmatrix}}_{n-2} = \underbrace{\begin{pmatrix} f_1 - u_0/(\Delta x)^2 \\ f_2 \\ \vdots \\ f_{n-3} \\ f_{n-2} - u_{n-1}/(\Delta x)^2 \end{pmatrix}}_{n-2}.$$

f_1 -= u[0]/dx/dx

f_{n-2} -= u[n-1]/dx/dx

1. Write a function which generates the reduced matrix system above, using *symmetric* matrix storage, for $n = 21$ and domain $[a, b] = [0, 2]$. Assume $\lambda = 1$.
2. ✓ Write a function which calculates the forcing function

   $f_i \Rightarrow x_i \Rightarrow x = x_0 + i dx = 0 + i\Delta x = i \cdot dx$

   $$f = -(\lambda + \pi^2)\sin(\pi x)$$

   $f = -(\lambda + \pi^2)\sin(\pi x_i) = -(\lambda + \pi^2)\sin(\pi \cdot i \cdot dx)$.

   on the grid points. The boundary conditions for this problem are $u(0) = u(2) = 0$.
3. ✓ Use the conjugate gradient implementation from Exercise 4.3 to solve the system of equations for u. Verify your solution matches the analytical solution of $u = \sin(\pi x)$.
4. ✓ Modify your code to solve for the forcing function

   $f_i = -(\lambda + \pi^2)\cos(\pi i dx)$

   $$f = -(\lambda + \pi^2)\cos(\pi x)$$

   on the domain $[a, b] = [0, 3]$ with boundary conditions $u(0) = 1$, $u(3) = -1$ and verify your solution matches the analytical solution of $u = \cos(\pi x)$.

$oodx2 = \frac{1}{(\Delta x)^2} = \beta.$

$\alpha = \frac{-2}{(\Delta x)^2} - \lambda = -2(oodx2) - \lambda.$

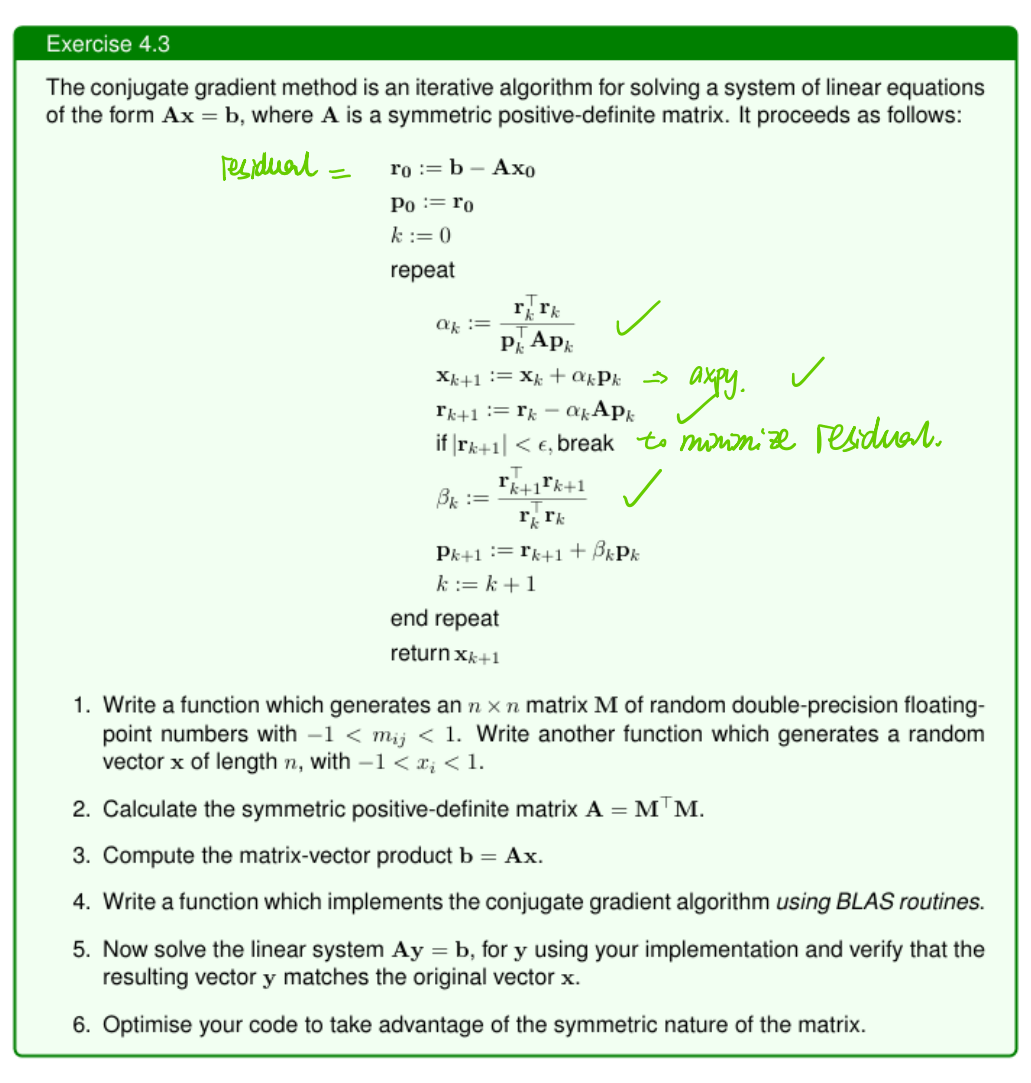
holmetz: $f = \frac{d^2 y}{dx^2} - \lambda u.$

$b = r_0$

$f = b - Ax_0.$

$r_0 = b - Ax_0.$

$y \Leftarrow \alpha A x + \beta y.$ ∴ $r = -Ax + r_0.$



Exercise 4.3

The conjugate gradient method is an iterative algorithm for solving a system of linear equations of the form $\mathbf{Ax} = \mathbf{b}$, where $\mathbf{A}$ is a symmetric positive-definite matrix. It proceeds as follows:

```
residual =  $\mathbf{r}_0 := \mathbf{b} - \mathbf{Ax}_0$
            $\mathbf{p}_0 := \mathbf{r}_0$
            $k := 0$
            repeat
                $\alpha_k := \frac{\mathbf{r}_k^\top \mathbf{r}_k}{\mathbf{p}_k^\top \mathbf{A} \mathbf{p}_k}$ ✓
                $\mathbf{x}_{k+1} := \mathbf{x}_k + \alpha_k \mathbf{p}_k$ → axpy. ✓
                $\mathbf{r}_{k+1} := \mathbf{r}_k - \alpha_k \mathbf{A} \mathbf{p}_k$ ✓
                if $|\mathbf{r}_{k+1}| < \epsilon$, break  to minimize residual.
                $\beta_k := \frac{\mathbf{r}_{k+1}^\top \mathbf{r}_{k+1}}{\mathbf{r}_k^\top \mathbf{r}_k}$ ✓
                $\mathbf{p}_{k+1} := \mathbf{r}_{k+1} + \beta_k \mathbf{p}_k$
                $k := k + 1$
            end repeat
            return $\mathbf{x}_{k+1}$
```

1. Write a function which generates an $n \times n$ matrix $\mathbf{M}$ of random double-precision floating-point numbers with $-1 < m_{ij} < 1$. Write another function which generates a random vector $\mathbf{x}$ of length n, with $-1 < x_i < 1$.
2. Calculate the symmetric positive-definite matrix $\mathbf{A} = \mathbf{M}^\top \mathbf{M}$.
3. Compute the matrix-vector product $\mathbf{b} = \mathbf{Ax}$.
4. Write a function which implements the conjugate gradient algorithm *using BLAS routines*.
5. Now solve the linear system $\mathbf{Ay} = \mathbf{b}$, for $\mathbf{y}$ using your implementation and verify that the resulting vector $\mathbf{y}$ matches the original vector $\mathbf{x}$.
6. Optimise your code to take advantage of the symmetric nature of the matrix.

dsymv(CblasRowMajor, ✓

CblasUpper, ✓

n. ✓

$\alpha = -1$ ✓

A. ✓

n. ✓

x ✓

1 ✓

1.0 ,

r, ✓

1) ✓

$$\frac{u(x_{i-1}) - 2u(x_i) + u(x_{i+1})}{(\Delta x)^2} - \lambda u(x_i) = f(x_i)$$

$$\frac{u(x_{i-1})}{(\Delta x)^2} - \left[\frac{2}{\Delta x^2} + \lambda\right] u(x_i) + \frac{u(x_{i+1})}{(\Delta x)^2} = f(x_i)$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ \frac{1}{(\Delta x)^2} & -\left[\frac{2}{\Delta x^2} + \lambda\right] & \frac{1}{(\Delta x)^2} & 0 \\ 0 & \frac{1}{(\Delta x)^2} & -\left[\frac{2}{(\Delta x)^2} + \lambda\right] & \frac{1}{\Delta x^2} \\ & & & \end{bmatrix} \begin{bmatrix} u_0 \\ u_1 \\ u_2 \\ \vdots \\ u_{n-1} \end{bmatrix} = \begin{bmatrix} f(x_0) \\ f(x_1) \\ f_2 \\ \vdots \\ f_{n-1} \end{bmatrix}$$

$u_0 = f(x_0).$

$u_{n-1} = f_{n-1}.$

A[18] = A[2×8+2] = A[i·lda+i].

i=1 ⇒ A[i·lda+i] = A[1(8)+1] = A[9].

i=2 ⇒ A[i·lda+i-1] = A[2(8)+2-1] = A[17]

$$\begin{array}{c|cccccccc|c|c} & H_0 & ⑧ & ⑯ & & & & & & & \\ 1 & \alpha & \beta & 0 & 0 & 0 & 0 & 0 & 0 & u_1 & f_1 \\ 2 & {}^{①}\beta & {}^{⑨}\alpha & {}^{⑰}\beta & 0 & 0 & 0 & 0 & 0 & u_2 & f_2 \\ 3 & {}^{②}0 & {}^{⑩}\beta & {}^{⑱}\alpha & \beta & 0 & 0 & 0 & 0 & u_3 & f_3 \\ 4 & {}^{③}0 & {}^{⑪}0 & \beta & \alpha & \beta & 0 & 0 & 0 & u_4 & f_4 \\ 5 & {}^{④}0 & {}^{⑫}0 & 0 & \beta & \alpha & \beta & 0 & 0 & u_5 & f_5 \\ 6 & {}^{⑤}0 & {}^{⑬}0 & 0 & 0 & \beta & \alpha & \beta & 0 & u_6 & f_6 \\ 7 & {}^{⑥}0 & {}^{⑭}0 & 0 & 0 & 0 & \beta & \alpha & \beta & u_7 & f_7 \\ 8 & {}^{⑦}0 & {}^{⑮}0 & 0 & 0 & 0 & 0 & \beta & \alpha & u_8 & f_8 \end{array}$$

main-diag = α.

Column Major:

A[0].

A[8+2] = A[10] = A[.

$u_0 = 0.$

$f_0 = -(\lambda + \pi^2)\sin(0) = 0.$

$u_0 = f_0,$

$\beta u_0 + \alpha u_1 + \beta u_2 = f_1$

$\Rightarrow \alpha u_1 + \beta u_2 = f_1 - \beta u_0.$

$\therefore f_1' = f_1 - \frac{u_0}{(\Delta x)^2}.$

$\alpha = \frac{-2}{(\Delta x)^2} - \lambda$

$\beta = \frac{1}{(\Delta x)^2}$

-30 16 -1

16 -30 16 -1

-1 16 -30 16 -1

BC: $u(0) = u(2) = 0.$

$\Rightarrow u(i=0) = u(i-1) = 0.$

u[0] = 0

u[n-1] = 0

H[20] = α.

```
#include <iostream>
#include <iomanip>
#include <cmath>
using namespace std;
#include "cblas.h"
/**
 * @brief Populate Helmholtz symmetric matrix (upper only)
 *
 * @param   nsv     Leading dimension of matrix
 * @param   H       Pointer to matrix storage of size nsv*nsv
 * @param   lam     Lambda coefficient
 * @param   dx      Grid spacing
 */
void FillHelmholtzMatrix(int nsv, double* H, double lam, double dx)
{
    const double oodx2 = 1.0/dx/dx;
    H[0] = -lam - 2.0*oodx2;
    for (int i = 1; i < nsv; ++i) {
        H[i*nsv + i - 1] = oodx2;
        H[i*nsv + i] = -lam - 2.0*oodx2;
    }
}
/**
 * @brief Fills the forcing vector with
 *        \f$ f = -(\lambda + \pi^2) \sin(\pi x) \f$
 *
 * @param   n       Vector dimension
 * @param   f       Pointer to vector storage of length n
 * @param   lam     Lambda coefficient
 * @param   dx      Grid spacing
 */
void FillForcingFunction1(int n, double* f, double lam, double dx)
{
    for (int i = 0; i < n; ++i) {
        f[i] = -(lam + M_PI*M_PI)*sin(M_PI*i*dx);
    }
}
/**
 * @brief Fills the forcing vector with
 *        \f$ f = -(\lambda + \pi^2) \cos(\pi x) \f$
 *
 * @param   n       Vector dimension
 * @param   f       Pointer to vector storage of length n
 * @param   lam     Lambda coefficient
 * @param   dx      Grid spacing
 */
void FillForcingFunction2(int n, double* f, double lam, double dx)
{
    for (int i = 0; i < n; ++i) {
        f[i] = -(lam + M_PI*M_PI)*cos(M_PI*i*dx);
    }
}
/**
 * @brief Enforces zero Dirichlet boundary conditions.
 *
 * @param   n       Vector dimension
 * @param   f       Pointer to forcing term storage of length n
 * @param   u       Pointer to solution vector storage of length n
 * @param   lam     Lambda coefficient
 * @param   dx      Grid spacing
 */
void EnforceBoundaryConditions1(int n, double* f, double* u, double
lam, double dx) {
    u[0] = sin(0);
    u[n-1] = sin(M_PI*(n-1)*dx);
    f[1] -= u[0]/dx/dx;
    f[n-2] -= u[n-1]/dx/dx;
}
/**
 * @brief Enforces zero Dirichlet boundary conditions.
 *
 * @param   n       Vector dimension
 * @param   f       Pointer to forcing term storage of length n
 * @param   u       Pointer to solution vector storage of length n
 * @param   lam     Lambda coefficient
 * @param   dx      Grid spacing
 */
void EnforceBoundaryConditions2(int n, double* f, double* u, double
lam, double dx) {
    u[0] = cos(0);
    u[n-1] = cos(M_PI*(n-1)*dx);
    f[1] -= u[0]/dx/dx;
    f[n-2] -= u[n-1]/dx/dx;
}
/**
 * @brief Computes the exact solution for first problem with
 *        \f$ u = \sin(\pi x) \f$
 *
 * @param   n       Vector dimension
 * @param   e       Pointer to solution vector storage of length n
 * @param   dx      Grid spacing
 */
void ExactSolution1(int n, double* e, double dx) {
    for (int i = 0; i < n; ++i) {
        e[i] = sin(M_PI*i*dx);
    }
}
/**
 * @brief Computes the exact solution for first problem with
 *        \f$ u = \cos(\pi x) \f$
 *
 * @param   n       Vector dimension
 * @param   e       Pointer to solution vector storage of length n
 * @param   dx      Grid spacing
 */
void ExactSolution2(int n, double* e, double dx) {
    for (int i = 0; i < n; ++i) {
        e[i] = cos(M_PI*i*dx);
    }
}
/**
 * @brief Prints a square matrix supplied in column-major full
storage.
 *
 * Note that for a symmetric matrix, only the upper diagonals or
lower
 * diagonals need to be stored.
 *
 * @param   nsv     Matrix dimension
 * @param   H       Pointer to matrix storage of size nsv*nsv
 */
void PrintMatrix(int nsv, double* H) {
    cout.precision(4);
    for (int i = 0; i < nsv; ++i) {
        for (int j = 0; j < nsv; ++j) {
            cout << setw(6) << H[j*nsv+i] << " ";
        }
        cout << endl;
    }
    cout << endl;
}
/**
 * @brief Prints a vector
 *
 * @param   n       Vector dimension
 * @param   u       Pointer to vector storage of length n
 * @param   dx      Grid spacing
 */
void PrintVector(int n, double* u, double dx) {
    for (int i = 0; i < n; ++i) {
        cout << i*dx << "  " << u[i] << endl;
    }
    cout << endl;
}

/**
 * @brief Solve the matrix problem \f$ Hu=f \f$ using the conjugate
gradient
 * algorithm.
 *
 * @param   nsv     Dimension of matrix system
 * @param   H       Pointer to full matrix storage of size nsv
containing
 *                  the symmetric matrix.
 * @param   f       Pointer to vector of length nsv containing
forcing for the
 *                  unknown degrees of freedom.
 * @param   u       Pointer to solution vector of length nsv, for
the unknown
 *                  degrees of freedom.
 */
void SolveConjugateGradient(int nsv, double* H, double* f, double*
u) {
    double* r = new double[nsv];
    double* p = new double[nsv];
    double* t = new double[nsv]; //temp
    int k;
    double alpha;
    double beta;
    double eps;
    double tol = 1e-08;
    cblas_dcopy(nsv, f, 1, r, 1);     // r_0 = b (i.e. f)
    cblas_dsymv(CblasColMajor, CblasUpper, nsv, -1.0, H, nsv,
                u, 1, 1.0, r, 1); // r_0 = b - A x_0
    cblas_dcopy(nsv, r, 1, p, 1);     // p_0 = r_0
    k = 0;
    do {
        cout << "Iteration " << k << endl;
        cblas_dsymv(CblasColMajor, CblasUpper, nsv, 1.0, H, nsv,
                    p, 1, 0.0, t, 1);
        alpha = cblas_ddot(nsv, t, 1, p, 1);
        alpha = cblas_ddot(nsv, r, 1, r, 1) / alpha; // compute
alpha_k
        beta  = cblas_ddot(nsv, r, 1, r, 1);
        cblas_daxpy(nsv, alpha, p, 1, u, 1);  // x_{k+1} = x_k +
alpha_k p_k
        cblas_daxpy(nsv, -alpha, t, 1, r, 1); // r_{k+1} = r_k -
alpha_k A p_k
        eps = cblas_dnrm2(nsv, r, 1);
        cout << "eps: " << eps << " tol=" << tol << endl;
        if (eps < tol) {
            break;
        }
        beta = cblas_ddot(nsv, r, 1, r, 1) / beta;
        cblas_dcopy(nsv, r, 1, t, 1);
        cblas_daxpy(nsv, beta, p, 1, t, 1);
        cblas_dcopy(nsv, t, 1, p, 1);
        k++;
    } while (k < 500);
    delete[] r;
    delete[] p;
    delete[] t;
}
/**
 * @brief Solves the Helmholtz problem for two different forcing
terms.
 */
int main() {
    const int    n   = 21;          // Number of grid-points
    const int    nsv = n - 2;       // Number of unknown DOFs
    const double lam = 1.0;         // Value of Lambda
    const double L   = 1.0;         // Length of domain
    const double dx  = L / (n - 1); // Grid-point spacing
    double* H = new double[nsv*nsv];// Helmholtz matrix storage
    double* u = new double[n];      // Solution vector
    double* f = new double[n];      // Forcing vector
    double* e = new double[n];      // Exact solution vector
    // Generate the Helmholtz matrix in symmetric conventional
storage.
    FillHelmholtzMatrix(nsv, H, lam, dx);
    cout << "Helmholtz matrix (symmetric): " << endl;
    PrintMatrix(nsv, H);
    // Problem 1 with f = -(lambda + pi^2) sin(pi x)
    FillForcingFunction1(n, f, lam, dx);
    cblas_dscal(n, 1.0, u, 1);      // x_0 = 0
    EnforceBoundaryConditions1(n, f, u, lam, dx);
    SolveConjugateGradient(nsv, H, f+1, u+1);
    cout << "Solution: " << endl;
    PrintVector(n, u, dx);
    cout << "Exact: " << endl;
    ExactSolution1(n, e, dx);
    PrintVector(n, e, dx);
    cblas_daxpy(n, -1.0, u, 1, e, 1);
    double err1 = cblas_dnrm2(n, e, 1);
    // Problem 2 with f = -(lambda + pi^2) cos(pi x)
    FillForcingFunction2(n, f, lam, dx);
    cblas_dscal(n, 0.0, u, 1);      // x_0 = 0
    EnforceBoundaryConditions2(n, f, u, lam, dx);
    SolveConjugateGradient(nsv, H, f+1, u+1);
    cout << "Solution: " << endl;
    PrintVector(n, u, dx);
    cout << "Exact: " << endl;
    ExactSolution2(n, e, dx);
    PrintVector(n, e, dx);
    cblas_daxpy(n, -1.0, u, 1, e, 1);
    double err2 = cblas_dnrm2(n, e, 1);
    // Print the resulting errors
    cout << "Error (problem 1): " << err1 << endl;
    cout << "Error (problem 2): " << err2 << endl;
    // Clean up
    delete[] H;
    delete[] u;
    delete[] f;
    delete[] e;
}
```